

C to LC-3 Assembly Language

One of the keys to learning how to write an assembly language is to understand how to map high level language constructs to equivalent assembly language. Here are the constructs described on this page.

- [Declaring Variables](#)
- [Simple Assignment](#)
- [Conditionals](#)
- [Compound Conditionals](#)
- [Loops](#)
- [Simple Array Access](#)
- [Advanced Array Access](#)

The registers used in the examples are arbitrary. The register you use may be any of the registers. However, you must not use any register that contains data you wish to use later, unless that data is also stored somewhere in memory.

Unlike most languages, LC-3 assembly is a case insensitive language. You can "spell" things in upper case, lower case, or mixed case. You may use different "spellings" in different places, but all refer to the same thing. For example, `blah`, `BLAH`, `Blah`, `BlaH` are all identical. The **only** place where case matters is in the `.STRINGZ` pseudo-op which is used to initialize memory with a quoted string (e.g. `.STRINGZ "Hello World"`). The case of the quoted string is preserved.

Declaring Variables

To declare a variable, you must give it a name and either initialize it or let it be initialized to 0. An array is declared by setting the number of elements. All variables declared as shown are global and live for the life of the program. You may have local variables that exist on the stack, but those are not covered in this document. There is no dynamic allocation (`malloc()/free()`) because no one has written routine(s) to do it.

C:

```
int a;           // simple variable (uninitialized)
int b = 2014;    // simple initialized variable
int c[10];       // array of 10 (uninitialized)
```

LC-3:

```
a    .BLKW 1      ; simple variable (or .FILL 0)
b    .FILL #2014  ; simple initialized variable
c    .BLKW 10     ; array of ten ints (initialized to 0)
d    .FILL foobar ; store address of foobar in variable d
```

Simple Assignment

The most basic operation is the assignment of one variable to another. Most modern computer architectures are load/store. This means that a value must be loaded from memory into a register, then written back to memory. There are no direct memory to memory copies. There may be additional instructions between the `LD` and `ST`.

C:

```
b = a;
```

LC-3:

```
LD R0, a ; load from memory to a register
ST R0, b ; store from register to memory
```

C:

```
b = a + 1;
```

LC-3:

```
LD R0, a      ; load from memory to a register
ADD R0, R0, #1 ; increment value
ST R0, b      ; store from register to memory
```

When using **LD/ST**, the location of the variable is fixed at assembly time and the location must be within 256 of the instruction. If either of these conditions is not met, pointers must be used.

Assignment Using Pointers

A pointer is a variable whose contents is the address of something else. This is most commonly another variable, but it may be the address of an instruction. Since a variable in LC-3 is 16 bits, a pointer can access any location in the LC-3's memory.

C:

```
pa = &a;
```

LC-3:

```
LEA R0, a ; get the address of the variable
ST R0, pa ; store it in the pointer variable
```

C:

```
b = *pa;
```

LC-3:

```
LDI R0, pa ; get the value at the address stored in pa
ST R0, b ; store it in b
```

C:

```
*pa = b;
```

LC-3:

```
LD R0, b ; load the value of b
STI R0, pa ; store it at the address stored in pa
```

Like the **LD/ST** instructions, the **LDI/STI/LEA** instructions have a limited range. The pointer variable itself must be within 256 of the instruction. The pointer can address anywhere in the memory.

Alternative Pointer Access

The **LDR/STR** instructions also access memory via pointers. However, the value of the pointer is stored in a register instead of a pointer variable. This is useful for accessing structures and variables on the stack. Consider a simple C structure that represents a date containing three integers representing a day, month, and year.

C:

```
typedef struct date {
    int day;
    int month;
    int year;
} date_t;
```

LC-3: No LC-3 equivalent (no struct type — allocate words manually) **C:**

```
date_t birthday;    // a date
date_t* birthday_ptr; // pointer to a date
```

LC-3:

```
birthday    .BLKW 3 ; a date is 3 words of memory
birthday_ptr .BLKW 1 ; a pointer is 1 word of memory
```

C:

```
birthday_ptr = &birthday;
```

LC-3:

```
LEA R0, birthday
ST  R0, birthday_ptr
```

C:

```
int d = birthday_ptr->day;
int m = birthday_ptr->month;
int y = birthday_ptr->year;
```

LC-3:

```
LD  R0, birthday_ptr
LDR R1, R0, #0    ; day is at offset 0
ST  R1, d         ; store in var d
LDR R1, R0, #1    ; month is at offset 1
ST  R1, m         ; store in var m
LDR R1, R0, #2    ; year is at offset 2
ST  R1, y         ; store in var y
```

Note how the address is stored in a register and then different offsets are used to access the different fields of a structure. In fact, C has a construct **offsetof()** that is used to compute the address of a field within a structure. In LC-3 assembly, you must compute the offsets yourself.

The **LDR/STR** instructions are also important in accessing method parameters and local variables. This is done using an offset from the frame pointer.

Conditionals

Conditionals allow different code to be executed depending on values encountered when the program is running. In C, we write logical expressions to compare values using a variety of symbols (`<`, `>`, `==`, `!=`, ...). In assembly language we can only test a value and determine if it is negative, zero, or positive. Therefore, the logical expressions in C must be converted to an operation with respect to zero.

C logical expr	C numeric expression	LC-3 branch	LC-3 negated branch
<code>if (a < b)</code>	<code>if ((a - b) < 0)</code>	BRn	BRzp
<code>if (a <= b)</code>	<code>if ((a - b) <= 0)</code>	BRnz	BRp
<code>if (a == b)</code>	<code>if ((a - b) == 0)</code>	BRz	BRnp
<code>if (a >= b)</code>	<code>if ((a - b) >= 0)</code>	BRzp	BRn
<code>if (a > b)</code>	<code>if ((a - b) > 0)</code>	BRp	BRnz
<code>if (a != b)</code>	<code>if ((a - b) != 0)</code>	BRnp	BRz
<code>if (a)</code>	<code>if (a != 0)</code>	BRnp	BRz
<code>if (!a)</code>	<code>if (a == 0)</code>	BRz	BRnp

Simple if

Converting a typical C condition is a two part process. First generate an LC-3 operation that sets the condition code in a way that can be used to make a decision. Then conditionally execute or skip some code.

Comparison in LC-3 often requires subtraction. But the LC-3 has no subtract operation, so one must change this to addition with a negated value (two's complement). The test must be negated so that the assembly code skips the body when the condition is false.

C:

```
if (a < b) {
    // do something
}
```

LC-3:

```
LD  R0, a      ; load a
LD  R1, b      ; load b
NOT R1, R1     ; begin 2's complement of b
ADD R1, R1, #1 ; R1 now has -b
ADD R0, R0, R1 ; R0 = a + (-b)
                ; condition code now set
BRzp SKIP     ; if false, skip over code

                ; code to do something (the true clause)

SKIP          ; remainder of code after if
```

Simple if/else

C:

```
if (a < b) {
    // do something
}
else {
    // do something else
}
```

LC-3:

```
LD R0, a      ; load a
LD R1, b      ; load b
NOT R1, R1    ; begin 2's complement of b
ADD R1, R1, #1 ; R1 now has -b
ADD R0, R0, R1 ; R0 = a + (-b)
               ; condition code now set
BRzp ELSE     ; if false, skip over code

               ; code to do something (the true clause)

BR END_ELSE   ; don't execute else code

ELSE          ; code for else clause here

END_ELSE      ; remainder of code after else
```

Conditions with Constants

Conditions often involve constants. When converting to assembly, store the **negative** value directly to avoid the extra NOT + ADD step.

Using positive constant (more steps):

```
NINETY .FILL #90      ; the constant 90
...
LD R0, score         ; load score
LD R1, NINETY         ; start two's complement
NOT R1, R1
ADD R1, R1, #1        ; R1 = -90
ADD R0, R0, R1        ; R0 = score + (-90)
BRn NOT_AN_A         ; didn't get an A
```

Using negative constant (fewer steps):

```
NNINETY .FILL #-90    ; the constant -90
...
LD R0, score         ; load score
LD R1, NNINETY       ; get -90
ADD R0, R0, R1        ; R0 = score + (-90)
BRn NOT_AN_A         ; didn't get an A
```

Compound Conditionals

Conditionals often have multiple conditions connected by logical and/or/not (`&&/||/!`). Code for each individual condition is required. In between are branch instructions that branch to the appropriate location as soon as the final result is known (short-circuit evaluation).

C:

```
if (a || (b && !c)) {  
    // do something  
}
```

LC-3:

```
LD    R0, a        ; load a  
BRnp  D0_IT        ; left clause of or is true, so execute  
                        ; get here if a is false  
LD    R0, b        ; load b  
BRz   SKIP         ; left side of and is false, so skip  
                        ; get here if b is true  
LD    R0, c        ; load c  
NOT   R0, R0       ; negate it (! operation)  
BRz   SKIP         ; right side of and is false, so skip  
  
D0_IT                ; code to do something (the true clause)  
  
SKIP                ; remainder of code after if
```

Loops

Loops combine a condition used to terminate the loop and an unconditional branch which causes the body of the loop to be repeated until the termination condition occurs.

for Loop

```
for (int i = 0; i < limit; i++) {
    // body of loop
}
```

Part	C	LC-3
init	<code>i = 0</code>	<code>AND R0, R0, #0</code> then <code>ST R0, i</code>
test	<code>i < limit</code>	load both, subtract, <code>BRzp END</code>
body	C code	LC-3 code
step	<code>i++</code>	load, <code>ADD R0, R0, #1</code> , store
repeat	—	<code>BR TEST</code>

LC-3:

```

AND R0, R0, #0 ; i = 0
ST R0, i

TEST LD R0, i ; get i
LD R1, limit ; get limit
NOT R1, R1 ; two's complement
ADD R1, R1, #1
ADD R0, R0, R1 ; compute i - limit
BRzp END ; done when (i - limit) >= 0

; LC-3 code for body

INCR LD R0, i ; get i
ADD R0, R0, #1 ; increment it
ST R0, i ; save the new value
BR TEST ; go back and test again

END ; code after loop completes
```

Note: First get it correct, then optimize (but only if necessary).

break / continue

- `break` → use `BRnzp END` inside the loop body
- `continue` → use `BRnzp INCR` inside the loop body

while Loop

A `for` loop is just a `while` loop with extra code. Remove the initialization and increment sections; the only code at the end of the loop is an unconditional branch (`BR TEST`) back to the termination test.

Count-Down Loop

If loop order is unimportant, counting down is more efficient — the decrement result can be tested directly without a separate subtraction.

```
for (int i = limit; i > 0; i--) {
    // body of loop
}
```

LC-3:

```
LD R0, limit    ; initialize loop counter

TEST BRnz END    ; done when i <= 0

                ; LC-3 code for body

DECR ADD R0, R0, #-1 ; decrement i
BR TEST          ; go back and test again

END              ; code after loop completes
```

If the body does not use `R0`, the counter can stay in a register the whole time — no `LD/ST` needed inside the loop.

do/while Loop

In a `do/while` loop the termination test occurs at the **end**, so the body executes at least once. Use the "positive" condition: if the condition is still true, branch back to the top.

```
do {
    // body of the loop
} while (i < limit);
```

LC-3:

```
D0_TOP          ; beginning of loop body

                ; LC-3 code for body

LD R0, i        ; get i
LD R1, limit     ; get limit
NOT R1, R1       ; two's complement
ADD R1, R1, #1
ADD R0, R0, R1   ; compute i - limit
BRn D0_TOP       ; continue while (i - limit) < 0
```


Simple Array Access

The following assumes each element of the array takes **one** word of memory. Array access requires the array and an index. Get the address of the beginning of the array, then compute the address of the i -th item by adding the index.

C:

```
b = c[i];
```

LC-3:

```
LEA R0, c      ; address of beginning of c
LD  R1, i      ; load index (i)
ADD R0, R0, R1 ; compute c + i (address of i-th element)
LDR R0, R0, #0 ; load c[i]
ST  R0, b      ; store it back (now b = c[i])
```

C:

```
c[i] = b;
```

LC-3:

```
LEA R0, c      ; address of beginning of c
LD  R1, i      ; load index (i)
ADD R0, R0, R1 ; compute c + i (address of i-th element)
LD  R1, b      ; load b
STR R1, R0, #0 ; store it back (now c[i] = b)
```

In C, array access `c[i]` is equivalent to pointer access `*(c + i)`.

Advanced Array Access

The correct way to do array access is to compute `c + i * sizeof(element)`. Each element takes the same amount of space, but the size may not be 1. C provides `sizeof()` for this; in LC-3 you must compute offsets yourself.

Example: Print an abbreviation of the *i*-th month (each entry is 4 bytes: 3 chars + null).

LC-3:

```
.ORIG x3000
LD  R0, I      ; get the index
ADD R0, R0, #-1 ; convert to 0-based index
ADD R0, R0, R0
ADD R0, R0, R0 ; R0 = 4*I (sizeof each entry is 4)
LEA R1, MONTHS ; R1 = address of beginning of array
ADD R0, R0, R1 ; R0 = address of the I-th element
PUTS                ; print out the abbreviation
HALT

I      .BLKW 1      ; the variable I (1-12)
MONTHS .STRINGZ "Jan"
      .STRINGZ "Feb"
      .STRINGZ "Mar"
      .STRINGZ "Apr"
      .STRINGZ "May"
      .STRINGZ "Jun"
      .STRINGZ "Jul"
      .STRINGZ "Aug"
      .STRINGZ "Sep"
      .STRINGZ "Oct"
      .STRINGZ "Nov"
      .STRINGZ "Dec"
      .END
```

Original by Fritz Sieker, Apr 2012